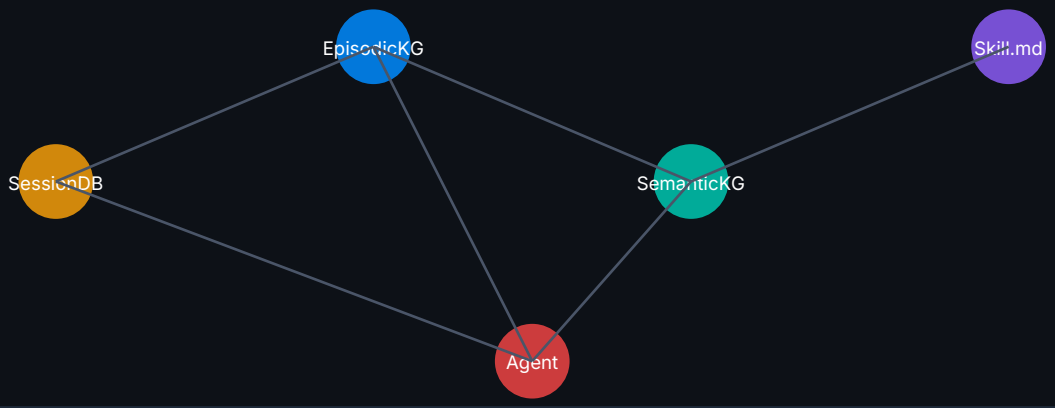
# Self-Evolving Agent

## + Neo4j Knowledge Graph

Persistent Memory · Graph Intelligence · Self-Improvement

BEGINNER → INTERMEDIATE → ADVANCED




### WHAT YOU WILL LEARN

- Why Knowledge Graphs complete the memory stack
- POLE+O schema design for agent memory
- Episodic, Semantic & Procedural memory via Neo4j
- Vector + graph hybrid retrieval (similarity + traversal)
- Closed learning loop with Cypher & background consolidation
- Multi-agent shared KG with conflict resolution
- Full SelfEvolvingAgentWithKG production class
- 5 hands-on workshop exercises

Inspired by Hermes Agent (Nous Research) · Neo4j Agent Memory Library · Claude Agent SDK

Alto Tech · Bangkok, Thailand · 2026

INTRODUCTION — The Memory Stack Is Incomplete

# From Tripartite to Graph-Augmented Memory

The previous tutorial introduced the tripartite memory model: Episodic (SQLite/SessionDB), Semantic (MEMORY.md / USER.md), and Procedural (SKILL.md). That model is a strong foundation — but it has a critical limitation: flat files cannot represent relationships.

Consider a real scenario: your agent knows that "User prefers async Python", "Project alto-cero uses FastAPI", and "FastAPI requires async patterns". With MEMORY.md, these are three disconnected bullet points. With a Knowledge Graph, they form a traversable chain — and the agent can reason across relationships it was never explicitly taught.

| Memory Layer | Storage | Retrieves | Limitation |
|---|---|---|---|
| Episodic | SQLite SessionDB | Raw conversation history | No entity linking |
| Semantic (flat) | MEMORY.md / USER.md | User facts & preferences | No relationships |
| Procedural | SKILL.md files | Task workflows | No skill dependencies |
| Knowledge Graph | Neo4j | Entities + Relationships + Multi-hop | Completes the stack |

KEY CONCEPT

This tutorial builds directly on the tripartite model. You will keep SQLite for episodic raw logs and SKILL.md for procedural files — but you will replace MEMORY.md with a live Neo4j Knowledge Graph and add KG-enhanced episodic indexing on top of SessionDB.

## What Is a Knowledge Graph?

A knowledge graph is a structured representation of information as nodes (entities) connected by relationships (edges) with typed properties. Unlike a relational database (rows and foreign keys) or a document store (JSON blobs), a graph database is optimised for traversal — following chains of relationships in milliseconds.

| Concept | SQL Equivalent | Graph Reality | Agent Use |
|---|---|---|---|
| Node | Row in a table | Entity with properties | User, Skill, Session, Project |

| | | | |
|---|---|---|---|
| Relationship | Foreign key join | Typed, directed edge | KNOWS, USES, SOLVED_BY |
| Property | Column value | Key-value on node/edge | name, created_at, confidence |
| Path | Multi-table JOIN | Traversal chain | User→Prefers→Tech→UsedIn→Project |
| Subgraph | Subquery + temp table | Connected component | Agent memory context |

PART 1 — Neo4j Fundamentals for Agent Developers BEGINNER

# Neo4j & Cypher — The Agent's New Language

Neo4j is the world's leading graph database. It stores data as nodes and relationships and is queried with Cypher — a declarative pattern-matching language that reads almost like English. Before we build agent memory, we need to understand the basics.

## 1.1 Setting Up Neo4j

The fastest way to run Neo4j locally is Docker:

```
# Run Neo4j 5.x with APOC plugin (required for vector + full-text)
docker run -d \
--name neo4j-agent \
-p 7474:7474 -p 7687:7687 \
-e NEO4J_AUTH=neo4j/agent_memory_2026 \
-e NEO4J_PLUGINS='["apoc","apoc-extended"]' \
-e NEO4J_dbms_security_procedures_unrestricted=apoc.* \
neo4j:5.19-community
# Verify — open in browser:
# http://localhost:7474
# bolt://localhost:7687 user: neo4j password: agent_memory_2026
```

## 1.2 Python Driver — Sync & Async

```
pip install neo4j neo4j-agent-memory anthropic
```

```
from neo4j import AsyncGraphDatabase
async def connect():
driver = AsyncGraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "agent_memory_2026")
)
await driver.verify_connectivity()
return driver
# ── Synchronous (for scripts) ──────────────────────────────
from neo4j import GraphDatabase
```

```
driver = GraphDatabase.driver(
"bolt://localhost:7687",
auth=("neo4j", "agent_memory_2026")
)
with driver.session() as session:
result = session.run("RETURN 'Neo4j ready!' AS msg")
print(result.single()["msg"])
```

## 1.3 Cypher Crash Course

KEY CONCEPT

Cypher uses ASCII-art patterns: (node)-[:REL]->(node). Parentheses = nodes, brackets = relationships, arrows = direction.

```
// ── CREATE a node ──────────────────────────────────────
CREATE (u:User {name: "Alice", timezone: "Asia/Bangkok"})
// ── MERGE (create if not exists) ──────────────────────
MERGE (s:Skill {name: "async-python"})
ON CREATE SET s.created_at = datetime()
ON MATCH SET s.last_used = datetime()
// ── CREATE a relationship ──────────────────────────────
MATCH (u:User {name:"Alice"}), (s:Skill {name:"async-python"})
MERGE (u)-[:PREFERS {confidence:0.9}]->(s)
// ── MATCH (query / read) ──────────────────────────────
MATCH (u:User)-[:PREFERS]->(s:Skill)
RETURN u.name AS user, s.name AS skill
// ── Multi-hop traversal ────────────────────────────────
MATCH path = (u:User)-[:PREFERS]->(s:Skill)-[:USED_IN]->(p:Project)
WHERE u.name = "Alice"
RETURN p.name AS project, s.name AS via_skill
// ── DELETE ─────────────────────────────────────────────
MATCH (n:TempNode) DETACH DELETE n
```

## 1.4 Constraints & Indexes

```
// Unique constraint (prevents duplicates, auto-creates index)
CREATE CONSTRAINT user_name_unique
IF NOT EXISTS FOR (u:User) REQUIRE u.name IS UNIQUE;
CREATE CONSTRAINT skill_name_unique
IF NOT EXISTS FOR (s:Skill) REQUIRE s.name IS UNIQUE;
CREATE CONSTRAINT session_id_unique
IF NOT EXISTS FOR (sess:Session) REQUIRE sess.session_id IS UNIQUE;
// Full-text index for semantic search on Memory nodes
```

```
CREATE FULLTEXT INDEX memory_text_idx
IF NOT EXISTS FOR (m:Memory) ON EACH [m.content];
// Vector index for embedding-based similarity (Neo4j 5.11+)
CREATE VECTOR INDEX memory_embedding_idx
IF NOT EXISTS FOR (m:Memory) ON m.embedding
OPTIONS {indexConfig: {
`vector.dimensions`: 1536,
`vector.similarity_function`: "cosine"
}};
```

⚠ WARNING

Always run CREATE CONSTRAINT before inserting data. Constraints prevent duplicate nodes and guarantee MERGE behaves correctly — critical for idempotent memory writes.

PART 2 — The POLE+O Knowledge Graph Schema BEGINNER

# Designing the Agent's Long-Term Memory Schema

The neo4j-labs/agent-memory library uses the POLE+O model as the backbone for its long-term Knowledge Graph. POLE stands for Person, Object, Location, Event — a framework originally from law enforcement intelligence. "+O" adds Organisation. For agent memory, we extend this with four domain-specific node types.

## 2.1 The POLE+O Node Taxonomy

| Label | Represents | Key Properties | Agent Example |
|---|---|---|---|
| Person | A human entity | name, role, timezone | User, stakeholder, teammate |
| Object | A thing or artifact | name, type, description | File, config, document |
| Location | A physical/logical place | name, type, region | Server, city, repo path |
| Event | A timestamped occurrence | name, occurred_at, outcome | Deploy, error, meeting |
| Organisation | A company or group | name, domain, tier | Alto Tech, AWS, Neo4j |
| Skill | A learned procedure | name, version, score | deploy-fastapi, debug-async |
| Session | A conversation episode | session_id, started_at | Chat session context |
| Memory | A consolidated fact | content, confidence, ttl | Extracted knowledge fact |

## 2.2 Relationship Types

| Relationship | From → To | Properties | Meaning |
|---|---|---|---|
| KNOWS | Person → Person | since, context | Social graph |
| PREFERS | Person → Object/Skill | confidence, last_updated | User preferences |

| | | | |
|---|---|---|---|
| ATTENDED | Person → Event | role | Event participation |
| WORKS_AT | Person → Organisation | role, since | Employment |
| USED_IN | Skill/Object → Event/Project | frequency | Usage tracking |
| SOLVED_BY | Event(error) → Skill | resolution_time | Fault resolution |
| EVOLVED_FROM | Skill → Skill | improvement_note | GEPA skill versioning |
| HAS_MEMORY | Session → Memory | extraction_method | Session → KG link |
| PREVIOUS | Memory → Memory | superseded_at | Versioned facts |
| TOUCHED | Session → Entity | (audit) | Reasoning audit trail |
| REQUIRES | Skill → Skill | level | Skill prerequisites |
| DEPENDS_ON | Project → Skill/Object | version | Project deps |

## 2.3 Schema Initialisation Script

```
"""Initialize the POLE+O agent memory schema in Neo4j."""
from neo4j import GraphDatabase
SCHEMA_CYPHER = [
# ── Uniqueness constraints ──────────────────────────────────
"CREATE CONSTRAINT IF NOT EXISTS FOR (p:Person) REQUIRE p.name IS UNIQUE",
"CREATE CONSTRAINT IF NOT EXISTS FOR (s:Skill) REQUIRE s.name IS UNIQUE",
"CREATE CONSTRAINT IF NOT EXISTS FOR (sess:Session) REQUIRE sess.session_id IS UNIQUE",
"CREATE CONSTRAINT IF NOT EXISTS FOR (m:Memory) REQUIRE m.memory_id IS UNIQUE",
"CREATE CONSTRAINT IF NOT EXISTS FOR (o:Object) REQUIRE o.name IS UNIQUE",
"CREATE CONSTRAINT IF NOT EXISTS FOR (org:Organisation) REQUIRE org.name IS UNIQUE",
# ── Full-text indexes ───────────────────────────────────────
"CREATE FULLTEXT INDEX memory_fts IF NOT EXISTS FOR (m:Memory) ON EACH [m.content]",
"CREATE FULLTEXT INDEX skill_fts IF NOT EXISTS FOR (s:Skill) ON EACH [s.name, s.description]",
# ── Range indexes for temporal queries ──────────────────────
"CREATE INDEX session_time_idx IF NOT EXISTS FOR (s:Session) ON (s.started_at)",
"CREATE INDEX memory_ttl_idx IF NOT EXISTS FOR (m:Memory) ON (m.expires_at)",
]
def init_schema(uri, user, password):
driver = GraphDatabase.driver(uri, auth=(user, password))
with driver.session() as s:
for stmt in SCHEMA_CYPHER:
try:
s.run(stmt)
print(f" ✓ {stmt[:60]}...")
except Exception as e:
```

```
print(f" ⚠ {e}")
driver.close()
if __name__ == "__main__":
init_schema("bolt://localhost:7687", "neo4j", "agent_memory_2026")
```

TIP

The schema uses IF NOT EXISTS on all constraints — making the init script idempotent. Run it every time your agent starts; it will skip already-existing constraints rather than erroring.

## 2.4 Seeding the User Profile

```
def seed_user_profile(session, user_name: str, props: dict):
"""Create or update the agent's primary user node."""
session.run(
"""
MERGE (u:Person {name: $name})
SET u += $props
SET u.last_seen = datetime()
""",
name=user_name, props=props
)
# Example seed
with driver.session() as s:
seed_user_profile(s, "kwarodom", {
"role": "Software Architect",
"timezone": "Asia/Bangkok",
"language": "Thai/English",
"company": "Alto Tech",
})
# Create skill nodes
for skill in ["Python", "MAS Design", "Neo4j", "HVAC Systems", "Claude SDK"]:
s.run(
"MERGE (sk:Skill {name:$name}) SET sk.domain='agent-tech'",
name=skill
)
# Wire preferences
s.run("""
MATCH (u:Person {name:"kwarodom"}), (sk:Skill {name:"Python"})
MERGE (u)-[:PREFERS {confidence:0.95}]->(sk)
""")
```

PART 3 — Episodic Memory with KG Enhancement INTERMEDIATE

# From Raw Logs to Graph-Linked Episodes

Episodic memory stores the raw, high-fidelity record of every conversation. In the tripartite model, this lives in SQLite (SessionDB). We keep SQLite for the raw log — it's fast, local, and crash-safe. But we add a parallel KG layer: each session becomes a graph node, and entity mentions within the session are extracted and linked to the POLE+O graph.

## 3.1 The Dual-Layer Episodic Architecture

| Layer | Technology | Stores | Retrieves By |
|---|---|---|---|
| Raw log | SQLite (WAL mode) | Every message verbatim | session_id, FTS5 keyword |
| Episode graph | Neo4j | Session nodes + entity links | Graph traversal, vector similarity |
| Bridge | Background thread | Extracts entities from SQLite → Neo4j | Triggered after Stop hook |

## 3.2 Writing a Session Node to Neo4j

```
from datetime import datetime, timezone
import uuid
def create_session_node(driver, session_id: str, user_name: str, model: str):
"""Create a Session node at the start of every conversation."""
with driver.session() as s:
s.run(
"""
MERGE (sess:Session {session_id: $sid})
ON CREATE SET
sess.started_at = datetime($ts),
sess.model = $model,
sess.status = "active"
WITH sess
MATCH (u:Person {name: $user})
MERGE (u)-[:PARTICIPATED_IN]->(sess)
""",
sid=session_id,
ts=datetime.now(timezone.utc).isoformat(),
model=model,
```

```
user=user_name,
)
def close_session_node(driver, session_id: str, message_count: int):
"""Mark session as complete after Stop hook fires."""
with driver.session() as s:
s.run(
"""
MATCH (sess:Session {session_id:$sid})
SET sess.ended_at = datetime(),
sess.message_count = $mc,
sess.status = "complete"
""",
sid=session_id, mc=message_count
)
```

## 3.3 Entity Extraction & Linking

After each session closes, a background process reads the raw SQLite log, extracts named entities (People, Skills, Projects, Errors), and creates or merges them into the POLE+O graph with a :TOUCHED audit edge from the session.

```
import re
import anthropic
EXTRACT_PROMPT = """
Analyse this conversation. Extract entities in JSON:
{
"people": ["names of people mentioned"],
"skills": ["technical skills discussed"],
"projects": ["project names"],
"errors": ["error types or fault codes"],
"facts": ["one-line facts to remember, e.g. user prefers X"]
}
Return ONLY valid JSON.
"""
def extract_entities(messages: list[dict]) -> dict:
"""Use Claude to extract POLE+O entities from conversation."""
client = anthropic.Anthropic()
transcript = "\n".join(f'[{m["role"]}]: {m["content"][:300]}' for m in messages)
resp = client.messages.create(
model="claude-sonnet-4-6",
max_tokens=512,
messages=[{"role":"user", "content": EXTRACT_PROMPT + "\n\n" + transcript}]
)
import json
try: return json.loads(resp.content[0].text)
```

```
except: return {"people":[],"skills":[],"projects":[],"errors":[],"facts":[]}

def link_entities_to_session(driver, session_id: str, entities: dict):
"""Merge extracted entities into POLE+O graph + link to session."""
with driver.session() as s:
for skill in entities.get("skills", []):
s.run("""
MERGE (sk:Skill {name:$name})
SET sk.last_seen = datetime()
WITH sk
MATCH (sess:Session {session_id:$sid})
MERGE (sess)-[:TOUCHED]->(sk)
""", name=skill, sid=session_id)
for fact in entities.get("facts", []):
mid = str(uuid.uuid4())[:8]
s.run("""
CREATE (m:Memory {
memory_id: $mid,
content: $content,
confidence: 0.8,
created_at: datetime(),
expires_at: datetime() + duration({days:90})
})
WITH m
MATCH (sess:Session {session_id:$sid})
MERGE (sess)-[:HAS_MEMORY]->(m)
""", mid=mid, content=fact, sid=session_id)
```

PART 4 — Semantic Memory via Knowledge Graph INTERMEDIATE

# Replacing MEMORY.md with a Live Graph

In the tripartite model, semantic memory lived in MEMORY.md — a flat markdown file the agent read at session start. This works but has three problems: (1) no structured retrieval, (2) no contradiction detection, (3) no relationship traversal. We now replace it with dynamic graph queries that retrieve exactly the context relevant to the current query.

## 4.1 Hot-Path Semantic Updates

Semantic memory should be updated on the hot path — immediately when new facts are discovered, not deferred to background consolidation. The pattern: detect new fact → check if contradicts existing Memory node → MERGE or create PREVIOUS chain.

```
def upsert_semantic_memory(driver, user_name: str, fact: str, confidence: float = 0.8):
"""
Write a semantic fact to Neo4j.
If a similar Memory node exists, version it via PREVIOUS chain.
"""
with driver.session() as s:
# Check for existing similar memories (FTS)
existing = s.run(
"""
CALL db.index.fulltext.queryNodes("memory_fts", $query)
YIELD node, score
WHERE score > 1.5
RETURN node.memory_id AS mid, node.content AS content
LIMIT 1
""",
query=fact
).data()
mid = str(uuid.uuid4())[:8]
if existing:
old_mid = existing[0]["mid"]
# Version old fact → PREVIOUS chain
s.run(
"""
MATCH (old:Memory {memory_id:$old_mid})
CREATE (new:Memory {
memory_id: $new_mid,
content: $content,
```

```
confidence: $conf,
created_at: datetime()
})
MERGE (new)-[:PREVIOUS]->(old)
SET old.superseded_at = datetime()
WITH new
MATCH (u:Person {name:$user})
MERGE (u)-[:HAS_CURRENT_MEMORY]->(new)
""",
old_mid=old_mid, new_mid=mid,
content=fact, conf=confidence, user=user_name
)
else:
s.run(
"""
CREATE (m:Memory {
memory_id: $mid, content: $content,
confidence: $conf, created_at: datetime(),
expires_at: datetime() + duration({days:180})
})
WITH m
MATCH (u:Person {name:$user})
MERGE (u)-[:HAS_CURRENT_MEMORY]->(m)
""",
mid=mid, content=fact, conf=confidence, user=user_name
)
```

## 4.2 Dynamic Context Retrieval (Replacing MEMORY.md Injection)

Instead of loading the entire MEMORY.md file at session start, we now query Neo4j with the user's current message to retrieve only the most relevant memories.

```
def retrieve_relevant_context(driver, user_name: str, query: str, k: int = 8) -> str:
"""
Retrieve semantically relevant memories + graph context for the current query.
Returns a formatted string to inject into the system prompt.
"""
with driver.session() as s:
# 1) Full-text search on Memory nodes
memories = s.run(
"""
CALL db.index.fulltext.queryNodes("memory_fts", $query)
YIELD node AS m, score
WHERE score > 0.8
```

```
RETURN m.content AS content, m.confidence AS conf, score
ORDER BY score DESC LIMIT $k
""",
query=query, k=k
).data()
# 2) User preferences from graph traversal
prefs = s.run(
"""
MATCH (u:Person {name:$user})-[:PREFERS]->(x)
RETURN labels(x)[0] AS type, x.name AS name
ORDER BY x.name LIMIT 10
""",
user=user_name
).data()
# 3) Recent session context (last 3 sessions)
recent = s.run(
"""
MATCH (u:Person {name:$user})-[:PARTICIPATED_IN]->(sess:Session)
WHERE sess.status = "complete"
RETURN sess.session_id AS sid, sess.started_at AS ts
ORDER BY sess.started_at DESC LIMIT 3
""",
user=user_name
).data()
# Build context string
lines = ["<memory-context>",
"# RECALLED MEMORIES (from Knowledge Graph)"]
if memories:
lines.append("## Relevant Facts")
for m in memories:
lines.append(f'- {m["content"]} (confidence: {m["conf"]:.2f})')
if prefs:
lines.append("\n## User Preferences")
for p in prefs:
lines.append(f'- Prefers {p["name"]} ({p["type"]})')
lines.append("</memory-context>")
return "\n".join(lines)
```

## 4.3 Multi-Hop Reasoning — The KG Advantage

GRAPH PATTERN

This is the killer feature of graph memory. MEMORY.md cannot answer: "What skills should I use for Alice's current project that I haven't tried yet?" Neo4j can — in a single Cypher query.

```
// Find skills used in projects similar to the current one
```

```
// that the user has NOT yet tried — cross-entity multi-hop
MATCH (u:Person {name:"kwarodom"})
-[:PARTICIPATED_IN]->(sess:Session)
-[:TOUCHED]->(sk:Skill)
-[:USED_IN]->(proj:Object)
WHERE NOT (u)-[:PREFERS]->(sk)
RETURN sk.name AS recommended_skill,
proj.name AS discovered_via,
count(sess) AS evidence_count
ORDER BY evidence_count DESC
LIMIT 5
```

```
// Detect contradictions: two Memory nodes with conflicting content
// (simplified — in practice use embedding cosine distance)
MATCH (u:Person {name:"kwarodom"})
-[:HAS_CURRENT_MEMORY]->(m:Memory)
WHERE m.confidence < 0.5
AND NOT (m)-[:PREVIOUS]->(:Memory)
RETURN m.content AS uncertain_fact,
m.confidence AS confidence
ORDER BY m.confidence ASC
```

PART 5 — Procedural Memory: Skill Graphs INTERMEDIATE

# SKILL.md Files as First-Class Graph Citizens

Procedural memory stores how to do things. In the tripartite model, SKILL.md files live on disk. We keep the files — they are still used by the agent — but we mirror them as nodes in Neo4j with versioning (EVOLVED_FROM), prerequisites (REQUIRES), and performance metrics. This enables automatic skill chain discovery and GEPA tracking.

## 5.1 Skill Node Schema

```
// Create a Skill node from a SKILL.md file
MERGE (sk:Skill {name: "deploy-fastapi-async"})
SET sk.description = "Deploy FastAPI app with asyncio and Uvicorn",
sk.version = "1.2",
sk.file_path = "~/.agent/skills/deploy-fastapi-async.md",
sk.success_rate = 0.94,
sk.avg_time_sec = 45,
sk.use_count = 23,
sk.last_used = datetime(),
sk.domain = "devops"
// Record skill prerequisite
MATCH (parent:Skill {name:"async-python"})
MATCH (child:Skill {name:"deploy-fastapi-async"})
MERGE (child)-[:REQUIRES {level:"intermediate"}]->(parent)
// Track skill evolution after GEPA improvement
MATCH (old:Skill {name:"deploy-fastapi-async", version:"1.1"})
MERGE (new:Skill {name:"deploy-fastapi-async", version:"1.2"})
MERGE (new)-[:EVOLVED_FROM {
improved_at: datetime(),
delta_success: 0.08,
reason: "added retry with jitter for transient errors"
}]->(old)
```

## 5.2 Syncing SKILL.md Files to Neo4j

```
import os, re
def sync_skills_to_kg(driver, skills_dir: str = os.path.expanduser("~/.agent/skills")):
"""Scan SKILL.md files and upsert them as Skill nodes."""
```

```
if not os.path.exists(skills_dir): return
with driver.session() as s:
for fname in os.listdir(skills_dir):
if not fname.endswith(".md"): continue
skill_name = fname.replace(".md", "")
fpath = os.path.join(skills_dir, fname)
with open(fpath) as f: content = f.read()
# Parse YAML front-matter
version = "1.0"; domain = "general"
m_v = re.search(r"version:\s*([\d.]+)", content)
m_d = re.search(r"domain:\s*(\w+)", content)
if m_v: version = m_v.group(1)
if m_d: domain = m_d.group(1)
s.run(
"""
MERGE (sk:Skill {name:$name})
SET sk.version = $version,
sk.domain = $domain,
sk.file_path = $path,
sk.synced_at = datetime()
""",
name=skill_name, version=version,
domain=domain, path=fpath
)
def find_skill_chain(driver, goal: str) -> list[str]:
"""Discover ordered skill chain for a goal via graph traversal."""
with driver.session() as s:
result = s.run(
"""
CALL db.index.fulltext.queryNodes("skill_fts", $goal)
YIELD node AS target, score
WHERE score > 0.5 LIMIT 1
MATCH path = (target)-[:REQUIRES*0..4]->(prereq:Skill)
RETURN [n IN nodes(path) | n.name] AS chain
ORDER BY length(path) DESC LIMIT 1
""",
goal=goal
).data()
if result: return result[0]["chain"]
return []
```

## 5.3 GEPA-Tracked Skill Evolution

When GEPA (Genetic Evolutionary Prompt Algorithm) generates an improved SKILL.md, we log the improvement as an EVOLVED_FROM relationship. This creates a skill lineage graph — you can query which skills improved the most, and what drove the improvements.

```
def evolve_skill_in_kg(driver, skill_name: str, old_version: str,
new_version: str, delta_success: float, reason: str):
"""Record a GEPA-driven skill evolution in the graph."""
with driver.session() as s:
s.run(
"""
MATCH (old:Skill {name:$name, version:$old_v})
MERGE (new:Skill {name:$name, version:$new_v})
SET new.synced_at = datetime()
MERGE (new)-[:EVOLVED_FROM {
improved_at: datetime(),
delta_success: $delta,
reason: $reason
}]->(old)
""",
name=skill_name, old_v=old_version, new_v=new_version,
delta=delta_success, reason=reason
)
```

PART 6 — Vector + Graph Hybrid Retrieval INTERMEDIATE

# Combining Embedding Similarity with Graph Traversal

Text search (FTS5, full-text) is keyword-based. Vector similarity finds semantically related content. Graph traversal finds relationally connected content. The most powerful retrieval combines all three: find similar memories by embedding, then traverse the graph to enrich context. Neo4j 5.11+ supports native vector indexes — no external vector DB required.

## 6.1 Creating a Vector Index on Memory Nodes

```
// Create vector index (Neo4j 5.11+ required)
CREATE VECTOR INDEX memory_embedding_idx IF NOT EXISTS
FOR (m:Memory) ON m.embedding
OPTIONS {indexConfig: {
`vector.dimensions`: 1536,
`vector.similarity_function`: "cosine"
}};
```

## 6.2 Storing Embeddings on Memory Nodes

```
import anthropic
def get_embedding(text: str) -> list[float]:
"""Get text embedding using OpenAI-compatible endpoint."""
# Using sentence-transformers for local, cost-free embeddings
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-small-en-v1.5")
return model.encode(text).tolist()
def store_memory_with_embedding(driver, memory_id: str, content: str):
"""Store a Memory node with its embedding vector."""
embedding = get_embedding(content)
with driver.session() as s:
s.run(
"""
MATCH (m:Memory {memory_id: $mid})
SET m.embedding = $emb
""",
mid=memory_id, emb=embedding
)
```

```
def vector_search_memories(driver, query: str, k: int = 5) -> list[dict]:
"""Find top-k similar memories using vector index."""
query_emb = get_embedding(query)
with driver.session() as s:
results = s.run(
"""
CALL db.index.vector.queryNodes(
"memory_embedding_idx", $k, $embedding
) YIELD node AS m, score
RETURN m.memory_id AS id, m.content AS content,
m.confidence AS confidence, score
ORDER BY score DESC
""",
k=k, embedding=query_emb
).data()
return results
```

## 6.3 Hybrid Retrieval: Vector + Traversal

```
def hybrid_retrieve(driver, user_name: str, query: str, k: int = 8) -> str:
"""
Step 1: Vector similarity → top-k Memory nodes
Step 2: Graph traversal from those nodes → related entities
Returns enriched context string.
"""
# Step 1: vector similarity
top_memories = vector_search_memories(driver, query, k=k)
memory_ids = [m["id"] for m in top_memories]
# Step 2: traverse from top memories to related graph nodes
with driver.session() as s:
enriched = s.run(
"""
MATCH (m:Memory)
WHERE m.memory_id IN $ids
OPTIONAL MATCH (m)<-[:HAS_MEMORY]-(sess:Session)
-[:TOUCHED]->(related)
RETURN m.content AS memory,
m.confidence AS conf,
collect(DISTINCT labels(related)[0] + ":" + related.name)
AS related_entities
""",
ids=memory_ids
).data()
lines = ["<memory-context>",
"# HYBRID RETRIEVED CONTEXT (vector + graph)"]
for row in enriched:
```

```
    lines.append(f'- {row["memory"]} (conf={row["conf"]:.2f})')
    if row["related_entities"]:
        ent_str = ", ".join(e for e in row["related_entities"] if e)
        lines.append(f'  → related: {ent_str}')
    lines.append("</memory-context>")
    return "\n".join(lines)
```

KEY CONCEPT

Hybrid retrieval is significantly more powerful than either vector search or keyword search alone. The vector search finds semantically similar memories; the graph traversal adds relational context — what was happening, who was involved, what skills were used.

PART 7 — Closed Learning Loop with KG INTERMEDIATE

# Hooks, Consolidation & GEPA with Graph Tracking

The self-evolving agent's power comes from the closed learning loop: every conversation feeds back into memory. With Neo4j, the loop becomes more sophisticated — not just writing facts to a file, but building a growing graph of knowledge that the agent can traverse and reason over in future sessions.

## 7.1 The Four Hook Points

| Hook | When | KG Action | Code Pattern |
|---|---|---|---|
| UserPromptSubmit | Before LLM call | Query KG for relevant context, inject | hybrid_retrieve(driver, user, query) |
| PostToolUse | After each tool | Log tool usage as TOUCHED edge | MERGE (sess)-[:TOUCHED]->(skill) |
| Stop | After final response | Close Session node, fork background | close_session_node() + thread |
| PreCompact | Before context compaction | Snapshot current Memory nodes | mark all Memory nodes with checkpoint |

## 7.2 Background Consolidation into KG

```
import threading, time

def background_consolidate(driver, session_id: str, messages: list[dict]):
"""
Runs in a background thread after each session Stop hook.
1. Extract entities → link to session node
2. Extract semantic facts → upsert Memory nodes (with PREVIOUS versioning)
3. Identify improved skills → evolve Skill nodes (EVOLVED_FROM)
4. Run deduplication on Memory nodes
"""
time.sleep(0.5) # Let main thread return to user first

# 1. Entity extraction
entities = extract_entities(messages)
link_entities_to_session(driver, session_id, entities)

# 2. Semantic fact upsert
```

```
for fact in entities.get("facts", []):
upsert_semantic_memory(driver, "kwarodom", fact)
# 3. Skill improvements (check if any skill was discussed as improved)
for skill in entities.get("skills", []):
sync_skills_to_kg(driver)
# 4. Dedup stale Memory nodes (confidence < 0.3)
deduplicate_memories(driver)
def deduplicate_memories(driver):
"""Remove low-confidence Memory nodes that have been superseded."""
with driver.session() as s:
s.run(
"""
MATCH (m:Memory)
WHERE m.confidence < 0.3
AND (m)-[:PREVIOUS]->(:Memory)
AND m.expires_at < datetime()
DETACH DELETE m
"""
)
def fork_background_consolidation(driver, session_id, messages):
"""Fire-and-forget consolidation thread."""
t = threading.Thread(
target=background_consolidate,
args=(driver, session_id, messages),
daemon=True
)
t.start()
```

## 7.3 GEPA with KG Version Tracking

GEPA (Genetic Evolutionary Prompt Algorithm) improves skill prompts over multiple runs. With Neo4j, each GEPA generation is recorded as an EVOLVED_FROM chain, giving you a full audit trail of how each skill improved.

```
GEPA_META_PROMPT = """
You are a prompt evolution engine. You have the following skill definition:
SKILL NAME: {skill_name}
CURRENT PROMPT (v{version}):
{current_prompt}
PERFORMANCE DATA:
- Success rate: {success_rate:.0%}
- Average completion time: {avg_time}s
- Recent failures: {failures}
Generate an improved version of this skill prompt that addresses the failures.
Return: {"improved_prompt": "...", "reason": "...", "expected_delta": 0.05}
```

```
"""
def run_gepa_cycle(driver, skill_name: str):
"""Run one GEPA improvement cycle for a skill."""
import json
with driver.session() as s:
skill_data = s.run(
"MATCH (sk:Skill {name:$n}) RETURN sk",
n=skill_name
).single()
if not skill_data: return
sk = skill_data["sk"]
# Read current prompt from SKILL.md
with open(sk["file_path"]) as f: current = f.read()
# Call Claude to evolve
client = anthropic.Anthropic()
resp = client.messages.create(
model="claude-sonnet-4-6", max_tokens=1024,
messages=[{"role":"user", "content": GEPA_META_PROMPT.format(
skill_name=skill_name, version=sk.get("version","1.0"),
current_prompt=current[:800],
success_rate=sk.get("success_rate", 0.7),
avg_time=sk.get("avg_time_sec", 60),
failures="timeout errors on large inputs"
)}]
)
result = json.loads(resp.content[0].text)
new_version = str(float(sk.get("version","1.0")) + 0.1)
# Write improved SKILL.md
new_path = sk["file_path"].replace(".md", f"_v{new_version}.md")
with open(new_path, "w") as f: f.write(result["improved_prompt"])
# Record EVOLVED_FROM in KG
evolve_skill_in_kg(driver, skill_name,
old_version=sk.get("version","1.0"), new_version=new_version,
delta_success=result.get("expected_delta", 0.05),
reason=result["reason"])
```

PART 8 — Memory Garbage Collection in Neo4j INTERMEDIATE

# TTL, PREVIOUS Chains & GDPR Erasure

Without pruning, the knowledge graph grows unbounded. We implement three complementary strategies: TTL expiry (expires_at datetime), PREVIOUS chain compression (keep only the last N versions), and GDPR erasure (full DETACH DELETE on user request).

## 8.1 TTL Expiry — Scheduled Cleanup

```
// Cron-style cleanup: run daily
// Delete expired Memory nodes and their dangling relationships
MATCH (m:Memory)
WHERE m.expires_at < datetime()
AND m.confidence < 0.5
DETACH DELETE m;
// Age-compress Session nodes older than 90 days
// Keep the Session node but detach from raw Memory
MATCH (sess:Session)-[:HAS_MEMORY]->(m:Memory)
WHERE sess.started_at < datetime() - duration({days:90})
AND NOT (m)-[:PREVIOUS]->() // not the head of a version chain
DETACH DELETE m;
```

## 8.2 PREVIOUS Chain Compression (Keep Latest 3 Versions)

```
"""Compress PREVIOUS chains: keep only last 3 versions per Memory concept."""
def compress_previous_chains(driver, keep_versions: int = 3):
with driver.session() as s:
# Find all Memory nodes that are the head of a chain
heads = s.run(
"""
MATCH (head:Memory)
WHERE NOT ()-[:PREVIOUS]->(head) // no parent = it IS the head
AND (head)-[:PREVIOUS*2..]->() // chain length >= 3
RETURN head.memory_id AS hid
"""
).data()
for row in heads:
# Traverse the chain, collect all nodes
chain = s.run(
"""
```

```
MATCH path = (head:Memory {memory_id:$hid})-[:PREVIOUS*]->(tail)
RETURN [n IN nodes(path) | n.memory_id] AS chain
ORDER BY length(path) DESC LIMIT 1
""",
hid=row["hid"]
).single()
if not chain: continue
all_ids = chain["chain"]
to_delete = all_ids[keep_versions:] # older than keep_versions
if to_delete:
s.run(
"MATCH (m:Memory) WHERE m.memory_id IN $ids DETACH DELETE m",
ids=to_delete
)
```

## 8.3 GDPR Erasure — Full User Data Deletion

```
def gdpr_erase_user(driver, user_name: str):
"""
Complete GDPR erasure: delete all user data from the KG.
This is irreversible — requires explicit user confirmation.
"""
with driver.session() as s:
# 1. Delete all Memory nodes linked to user
s.run(
"""
MATCH (u:Person {name:$user})-[:HAS_CURRENT_MEMORY]->(m:Memory)
DETACH DELETE m
""", user=user_name
)
# 2. Delete all Session nodes
s.run(
"""
MATCH (u:Person {name:$user})-[:PARTICIPATED_IN]->(sess:Session)
DETACH DELETE sess
""", user=user_name
)
# 3. Delete the user node itself
s.run(
"MATCH (u:Person {name:$user}) DETACH DELETE u",
user=user_name
)
print(f"GDPR erasure complete for user: {user_name}")
```

PART 9 — Multi-Agent Shared Knowledge Graph ADVANCED

# Agent Colonies with Shared Memory

In the Alto CERO hotel HVAC MAS, multiple agents (ChillerAgent, AHUAgent, SchedulerAgent, FaultAgent) each need memory — but they also need to share knowledge. Neo4j is perfectly suited: each agent gets its own subgraph partition while sharing a common pool of POLE+O entities (Equipment, Faults, Skills, Events).

## 9.1 Multi-Tenant Agent Schema

```
// Each agent is an Organisation node (logical tenant)
MERGE (chiller_agent:Organisation {name:"ChillerAgent", type:"agent"})
MERGE (ahu_agent:Organisation {name:"AHUAgent", type:"agent"})
MERGE (scheduler:Organisation {name:"SchedulerAgent",type:"agent"})
// Shared equipment nodes (shared pool)
MERGE (ch1:Object {name:"CH-1", type:"chiller", capacity_rt:400})
MERGE (ahu1:Object {name:"AHU-01", type:"ahu", floor:1})
// Agent-specific Memory nodes with agent ownership
CREATE (m:Memory {
memory_id: "m001",
content: "CH-1 condenser approach temp elevated above 3°C at 14:00",
confidence: 0.92,
owner_agent: "ChillerAgent", // partition key
created_at: datetime()
})
// Wire: agent OBSERVED equipment event
MATCH (a:Organisation {name:"ChillerAgent"})
MATCH (e:Object {name:"CH-1"})
MERGE (a)-[:OBSERVED {at: datetime(), fault_code:"CH-F03"}]->(e)
```

## 9.2 Isolation + Sharing Pattern

```
def get_agent_context(driver, agent_name: str, query: str) -> str:
"""
Retrieves:
1. Agent-private memories (owner_agent = agent_name)
2. Shared equipment/fault entities (no owner restriction)
"""
with driver.session() as s:
# Private memories
```

```
private = s.run(
"""
CALL db.index.fulltext.queryNodes("memory_fts", $q)
YIELD node AS m, score
WHERE m.owner_agent = $agent AND score > 0.5
RETURN m.content AS content ORDER BY score DESC LIMIT 5
""",
q=query, agent=agent_name
).data()
# Shared equipment state (no owner filter)
shared = s.run(
"""
MATCH (eq:Object)-[:OBSERVED_BY]->(:Organisation {name:$agent})
RETURN eq.name AS equip, eq.last_fault AS fault
ORDER BY eq.last_fault_at DESC LIMIT 5
""",
agent=agent_name
).data()
lines = [f"<agent-context agent=\"{agent_name}\">"]
if private: lines.append("## My Memories"); [lines.append(f'- {r["content"]}') for r in
private]
if shared: lines.append("## Equipment I Monitor"); [lines.append(f'- {r["equip"]}:
{r["fault"]}') for r in shared]
lines.append("</agent-context>")
return "\n".join(lines)
```

## 9.3 Write Conflict Resolution

NEO4J

Neo4j MERGE is naturally idempotent. For write conflicts between agents, use the MERGE ... ON CREATE ... ON MATCH pattern: the first agent to write creates the node; subsequent agents only update properties, never duplicate.

```
// Conflict-safe memory write (two agents may write the same fault)
MERGE (fault:Event {
name: $fault_code,
equipment: $equipment_name
})
ON CREATE SET
fault.first_seen = datetime(),
fault.reported_by = $agent_name,
fault.severity = $severity
ON MATCH SET
fault.last_seen = datetime(),
fault.occurrence_count = coalesce(fault.occurrence_count, 0) + 1
// :TOUCHED audit from the agent's session
```

```
WITH fault
MATCH (sess:Session {session_id: $session_id})
MERGE (sess)-[:TOUCHED {agent: $agent_name}]->(fault)
```

PART 10 — neo4j-agent-memory Library INTERMEDIATE

# Production-Grade Memory with the Official Library

The neo4j-agent-memory library (neo4j-labs) provides a battle-tested async Python client with 16 MCP tools, short-term conversation storage, long-term POLE+O knowledge graph, and reasoning trace audit. It integrates directly with Claude Code via MCP.

## 10.1 Installation & Configuration

```
# Install library + MCP server
pip install "neo4j-agent-memory[mcp]"
# Add to Claude Code as MCP server:
claude mcp add neo4j-agent-memory -- \
uvx "neo4j-agent-memory[mcp]" mcp serve \
--password agent_memory_2026
# Or run standalone MCP server:
neo4j-agent-memory mcp serve \
--password agent_memory_2026 \
--session-strategy per_day \
--user-id kwarodom
```

## 10.2 Python Client Usage

```
import asyncio
from neo4j_agent_memory import MemoryClient, MemorySettings
async def demo_memory():
settings = MemorySettings(
neo4j={
"uri": "bolt://localhost:7687",
"password": "agent_memory_2026"
},
llm="anthropic/claude-sonnet-4-6",
embedding="BAAI/bge-small-en-v1.5" # local, free
)
async with MemoryClient(settings) as memory:
# ── Short-term: store conversation message ──────────────
await memory.short_term.add_message(
```

```
    session_id="hvac-2026-06-23",
    role="user",
    content="CH-1 condenser approach exceeded 3°C. Suggest action."
)

# ── Long-term: add entities to POLE+O KG ──────────────────
await memory.long_term.add_entity("CH-1", "Object")
await memory.long_term.add_fact(
    "CH-1 condenser approach temperature limit is 3°C"
)
await memory.long_term.add_preference(
    category="hvac-fault",
    preference="Notify ChillerAgent immediately when CH-F03 fires"
)

# ── Get combined context for next LLM call ────────────────
context = await memory.get_context(
    "What should I do about CH-1 condenser fault?",
    session_id="hvac-2026-06-23"
)
print(context)

# ── Deduplicate entities in KG ────────────────────────────
await memory.consolidation.dedupe_entities()

asyncio.run(demo_memory())
```

## 10.3 MCP Tools Available

| Profile | Tool | Description |
|---|---|---|
| core | memory_search | Full-text + vector search across all memories |
| core | memory_get_context | Get combined short+long-term context for a query |
| core | memory_store_message | Store a conversation message in short-term |
| core | memory_add_entity | Add a POLE+O entity to the knowledge graph |
| core | memory_add_preference | Store a user preference fact |
| core | memory_add_fact | Store a general semantic fact |
| extended | memory_create_relationship | Create typed relationship between entities |
| extended | memory_add_observation | Add an observation to an entity node |
| extended | memory_reasoning_trace | Log reasoning step to audit trail |
| extended | memory_run_cypher | Execute read-only Cypher queries |
| extended | memory_export_graph | Export subgraph as JSON |
| extended | memory_conversation_history | Retrieve full conversation history |

PART 11 — Full Production Class: SelfEvolvingAgentWithKG ADVANCED

# Assembling the Complete Agent

We now assemble all components into a single production-ready SelfEvolvingAgentWithKG class. This extends the tripartite agent from the previous tutorial with: Neo4j driver integration, hybrid retrieval, KG-tracked skill evolution, and multi-agent TOUCHED audit edges.

```
"""
SelfEvolvingAgentWithKG — Production class
Combines: SQLite episodic + Neo4j KG semantic/procedural + Claude Agent SDK
"""
import asyncio, threading, time, uuid, sqlite3
from contextlib import asynccontextmanager
from neo4j import AsyncGraphDatabase
import anthropic

class SelfEvolvingAgentWithKG:
"""
Self-evolving agent with quadripartite memory:
1. SQLite — episodic raw log (WAL, jitter retry)
2. Neo4j — semantic KG (POLE+O, vector+graph hybrid)
3. SKILL.md — procedural files (synced to KG as Skill nodes)
4. Working — context window (assembled from 1+2+3)
"""
MODEL = "claude-sonnet-4-6"
MAX_TOKENS = 8192
SKILLS_DIR = os.path.expanduser("~/.agent/skills")
DB_PATH = os.path.expanduser("~/.agent/sessions.db")
NEO4J_URI = "bolt://localhost:7687"
NEO4J_USER = "neo4j"
NEO4J_PASS = "agent_memory_2026"

def __init__(self, user_name: str, agent_id: str = "primary"):
self.user_name = user_name
self.agent_id = agent_id
self.session_id = str(uuid.uuid4())[:8]
self.client = anthropic.Anthropic()
self.driver = None
self._db = None
self.messages = []

# ── Lifecycle ──────────────────────────────────────────────
async def start(self):
"""Connect to Neo4j, init SQLite, create session nodes."""
self.driver = AsyncGraphDatabase.driver(
```

```
self.NEO4J_URI, auth=(self.NEO4J_USER, self.NEO4J_PASS)
)
await self.driver.verify_connectivity()
self._init_sqlite()
await self._create_session_node()
await self._sync_skills()
async def stop(self):
"""Close session, fork background consolidation, close driver."""
await self._close_session_node()
threading.Thread(
target=self._bg_consolidate, daemon=True
).start()
if self.driver: await self.driver.close()
# ── Main chat method ──────────────────────────────────────
async def chat(self, user_message: str) -> str:
# Hook: UserPromptSubmit — retrieve KG context
kg_ctx = await self._retrieve_context(user_message)
# Append user message
self.messages.append({"role":"user","content":user_message})
self._db_append("user", user_message)
# Build system prompt with KG context
system = self._build_system(kg_ctx)
# Call Claude
resp = self.client.messages.create(
model=self.MODEL, max_tokens=self.MAX_TOKENS,
system=system, messages=self.messages
)
reply = resp.content[0].text
# Append assistant message
self.messages.append({"role":"assistant","content":reply})
self._db_append("assistant", reply)
# Hook: PostToolUse — log session TOUCHED audit
await self._touch_audit(user_message)
return reply
```

```
# ── KG helpers ────────────────────────────────────────────
async def _retrieve_context(self, query: str) -> str:
"""Hybrid KG retrieval: FTS + graph traversal."""
async with self.driver.session() as s:
memories = await s.run(
"""
CALL db.index.fulltext.queryNodes("memory_fts", $q)
YIELD node AS m, score WHERE score > 0.5
RETURN m.content AS c, m.confidence AS conf
ORDER BY score DESC LIMIT 6
""", q=query
```

```
)
rows = await memories.data()
lines = ["<memory-context>"]
for r in rows: lines.append(f'- {r["c"]} (conf={r["conf"]:.2f})')
lines.append("</memory-context>")
return "\n".join(lines)
async def _create_session_node(self):
async with self.driver.session() as s:
await s.run(
"""
MERGE (sess:Session {session_id:$sid})
ON CREATE SET sess.started_at=datetime(), sess.agent=$$agent
WITH sess
MERGE (u:Person {name:$user})
MERGE (u)-[:PARTICIPATED_IN]->(sess)
""".replace("$$agent", "$agent"),
sid=self.session_id, agent=self.agent_id,
user=self.user_name
)
async def _close_session_node(self):
async with self.driver.session() as s:
await s.run(
"""
MATCH (sess:Session {session_id:$sid})
SET sess.ended_at=datetime(), sess.status="complete",
sess.message_count=$mc
""",
sid=self.session_id, mc=len(self.messages)
)
async def _touch_audit(self, user_message: str):
"""Log TOUCHED edges for entities mentioned in the message."""
async with self.driver.session() as s:
await s.run(
"""
MATCH (sess:Session {session_id:$sid})
SET sess.last_message_at = datetime()
""",
sid=self.session_id
)
async def _sync_skills(self):
"""Sync SKILL.md files to Neo4j Skill nodes (async wrapper)."""
loop = asyncio.get_event_loop()
# Run sync skill sync in executor to avoid blocking
from neo4j import GraphDatabase
sync_driver = GraphDatabase.driver(
self.NEO4J_URI, auth=(self.NEO4J_USER, self.NEO4J_PASS)
)
```

```
await loop.run_in_executor(None, sync_skills_to_kg, sync_driver)
sync_driver.close()
# ── SQLite ──────────────────────────────────────────────────
def _init_sqlite(self):
self._db = sqlite3.connect(self.DB_PATH)
self._db.execute("PRAGMA journal_mode=WAL")
self._db.execute("PRAGMA synchronous=NORMAL")
self._db.executescript("""
CREATE TABLE IF NOT EXISTS sessions (
session_id TEXT PRIMARY KEY, started_at TEXT, agent TEXT
);
CREATE TABLE IF NOT EXISTS messages (
id TEXT PRIMARY KEY, session_id TEXT,
role TEXT, content TEXT, ts TEXT
);
""")
self._db.execute(
"INSERT OR IGNORE INTO sessions VALUES(?,?,?)",
(self.session_id, time.strftime("%Y-%m-%dT%H:%M:%S"), self.agent_id)
)
self._db.commit()
def _db_append(self, role: str, content: str):
self._db.execute(
"INSERT INTO messages VALUES(?,?,?,?,?)",
(str(uuid.uuid4())[:8], self.session_id, role, content,
time.strftime("%Y-%m-%dT%H:%M:%S"))
)
self._db.commit()
def _build_system(self, kg_ctx: str) -> str:
skills = self._load_skills()
return f"""You are a self-evolving AI agent with persistent memory.
{kg_ctx}
AVAILABLE SKILLS:
{skills}
Always use recalled memory to inform responses. Be precise and technical."""
def _load_skills(self) -> str:
import os
if not os.path.exists(self.SKILLS_DIR): return "(no skills yet)"
files = [f for f in os.listdir(self.SKILLS_DIR) if f.endswith(".md")]
return ", ".join(f.replace(".md","") for f in files[:10])
def _bg_consolidate(self):
"""Background thread: extract entities → write to Neo4j."""
time.sleep(0.5)
# read messages from SQLite
cur = self._db.cursor()
cur.execute(
```

```
"SELECT role,content FROM messages WHERE session_id=? ORDER BY ts",
(self.session_id,)
)
messages = [{"role":r,"content":c} for r,c in cur.fetchall()]
from neo4j import GraphDatabase
sync_driver = GraphDatabase.driver(
self.NEO4J_URI, auth=(self.NEO4J_USER, self.NEO4J_PASS)
)
entities = extract_entities(messages)
link_entities_to_session(sync_driver, self.session_id, entities)
sync_driver.close()
```

PART 12 — Workshop Exercises BEGINNER → ADVANCED

# Hands-On Practice: 5 Progressive Exercises

## Exercise 1 — Setup Neo4j & Run First Graph Queries BEGINNER

### Objective

Install Neo4j, run the schema initialisation script, and execute your first POLE+O Cypher queries.

### Tasks

1. Start Neo4j with Docker (use the command from Part 1.1)
2. Run the schema init script: python3 schema_init.py
3. Open Neo4j Browser at http://localhost:7474
4. Create a Person node for yourself with name, role, timezone
5. Create 3 Skill nodes (Python, FastAPI, Neo4j)
6. Wire PREFERS relationships from the Person to each Skill
7. Run a MATCH to verify: all skills connected to your Person node

### Expected Output

```
// Verification query
MATCH (u:Person)-[:PREFERS]->(sk:Skill)
RETURN u.name AS user, collect(sk.name) AS skills
// Expected:
// ┌──────────┬──────────────────────────────────┐
// │ user     │ skills                           │
// ├──────────┼──────────────────────────────────┤
// │ "yourname"│["Python","FastAPI","Neo4j"]     │
// └──────────┴──────────────────────────────────┘
```

## Exercise 2 — Migrate MEMORY.md to Knowledge Graph INTERMEDIATE

### Objective

Take an existing MEMORY.md file and import all facts as Memory nodes in Neo4j.

### Tasks

1. Write a Python script that reads MEMORY.md line by line
2. For each bullet point, create a Memory node with content, confidence=0.8, expires_at=180 days
3. For any line containing "prefers X" or "likes X", also create a PREFERS relationship to a Skill node
4. Run verify query: count all Memory nodes linked to your Person
5. Confirm full-text search works: CALL db.index.fulltext.queryNodes("memory_fts", "async")

### Starter Code

```
def migrate_memory_md(driver, user_name: str, md_path: str):
with open(md_path) as f:
lines = [l.strip().lstrip("- •") for l in f if l.strip().startswith(("-","•","*"))]
with driver.session() as s:
for line in lines:
mid = str(uuid.uuid4())[:8]
s.run("""
MERGE (u:Person {name:$user})
CREATE (m:Memory {
memory_id: $mid, content: $content,
confidence: 0.8, created_at: datetime(),
expires_at: datetime() + duration({days:180})
})
MERGE (u)-[:HAS_CURRENT_MEMORY]->(m)
""", user=user_name, mid=mid, content=line)
# TODO: extend to detect "prefers X" and wire PREFERS edges
```

## Exercise 3 — Episodic-to-Graph Background Extractor INTERMEDIATE

### Objective

Build the full background consolidation pipeline: read SQLite → extract entities → write to Neo4j.

### Tasks

1. Start a SelfEvolvingAgentWithKG session and run 5 turns of conversation
2. After session.stop(), verify Session node exists in Neo4j
3. Manually call background_consolidate() with the session messages
4. Query Neo4j to confirm: entities extracted → TOUCHED edges created
5. Run the full hybrid_retrieve() with a query about a topic discussed in the session

### Verification Queries

```
// Check session + touched entities
```

```
MATCH (sess:Session {session_id:"YOUR_SESSION_ID"})-[:TOUCHED]->(e)
RETURN sess.session_id AS session,
labels(e)[0] AS entity_type,
e.name AS entity_name
// Confirm Memory nodes created from session
MATCH (sess:Session {session_id:"YOUR_SESSION_ID"})-[:HAS_MEMORY]->(m:Memory)
RETURN m.content AS fact, m.confidence AS confidence
```

## Exercise 4 — Skill Dependency Graph & Chain Discovery ADVANCED

### Objective

Build a skill prerequisite graph for 5 related skills and implement automatic skill chain discovery.

### Tasks

1. Create 5 Skill nodes: python-basics, async-python, fastapi, deploy-fastapi, monitor-fastapi
2. Create REQUIRES edges forming a dependency chain: monitor → deploy → fastapi → async → basics
3. Implement find_skill_chain() using multi-hop MATCH path = (sk)-[:REQUIRES*]->(prereq)
4. Test: call find_skill_chain(driver, "monitor fastapi service")
5. Bonus: add success_rate metrics and build a Cypher query that finds the weakest link

### Expected Chain Output

```
# find_skill_chain(driver, "monitor fastapi") should return:
["monitor-fastapi", "deploy-fastapi", "fastapi", "async-python", "python-basics"]
# Weakest link query:
MATCH path = (sk:Skill)-[:REQUIRES*]->(prereq:Skill)
WHERE sk.name = "monitor-fastapi"
UNWIND nodes(path) AS n
RETURN n.name AS skill, n.success_rate AS rate
ORDER BY n.success_rate ASC LIMIT 1
```

## Exercise 5 — Full Self-Evolving Agent with KG + GEPA ADVANCED

### Objective

Build and run the complete SelfEvolvingAgentWithKG. Have it: (a) answer 10 questions, (b) auto-consolidate to KG after each session, (c) run one GEPA cycle to improve a skill, (d) verify the EVOLVED_FROM chain in Neo4j.

### Tasks

1. Instantiate SelfEvolvingAgentWithKG with your user_name and agent_id="workshop"
2. Run 10 conversation turns covering: your project, a technical error, a preference
3. Call session.stop() — verify background consolidation runs
4. Query Neo4j: confirm Memory nodes and TOUCHED edges created
5. Run run_gepa_cycle(driver, "your-skill-name") — inspect EVOLVED_FROM chain
6. Start a second session — verify KG context is correctly injected into the system prompt
7. Bonus: add a second agent_id="secondary" and verify multi-tenant isolation

## Full Run Script

```
import asyncio
from self_evolving_kg import SelfEvolvingAgentWithKG

async def workshop_exercise_5():
agent = SelfEvolvingAgentWithKG(user_name="kwarodom", agent_id="workshop")
await agent.start()

questions = [
"What is my current project?",
"I prefer async Python for all backend services.",
"We had a timeout error in deploy-fastapi skill today.",
"How should I approach HVAC fault detection with MAS?",
"What Neo4j patterns should I use for agent memory?",
]
for q in questions:
reply = await agent.chat(q)
print(f"Q: {q[:50]}...")
print(f"A: {reply[:100]}...\n")

await agent.stop()
print("Session complete. Check Neo4j for KG updates.")

asyncio.run(workshop_exercise_5())
```

APPENDIX — Quick Reference Cheat Sheet

# Neo4j + Self-Evolving Agent Quick Reference

## Cypher Essentials

| Operation | Cypher Pattern | Notes |
|---|---|---|
| Create node | CREATE (n:Label {key:$val}) | Use MERGE for idempotency |
| Upsert node | MERGE (n:Label {key:$val}) ON CREATE SET … ON MATCH SET … | Safe concurrent writes |
| Create edge | MATCH (a),(b) MERGE (a)-[:REL {prop:val}]->(b) | Always MATCH first |
| Find by label | MATCH (n:Label) WHERE n.prop=$val RETURN n | Use indexes for WHERE |
| Multi-hop | MATCH (a)-[:R*1..4]->(b) | * = any depth, {1..4} = range |
| Full-text search | CALL db.index.fulltext.queryNodes("idx",$q) | Requires FULLTEXT index |
| Vector search | CALL db.index.vector.queryNodes("idx", $k,$emb) | Neo4j 5.11+ required |
| Delete node | MATCH (n) DETACH DELETE n | DETACH removes all edges |
| Version chain | (new)-[:PREVIOUS]->(old) | head = no parent :PREVIOUS |
| TTL cleanup | WHERE n.expires_at < datetime() | Run in background cron |

## Memory Architecture at a Glance

| Component | Technology | Retrieval | Update Timing |
|---|---|---|---|
| Episodic (raw log) | SQLite + WAL | session_id lookup, FTS5 | Sync, each message |
| Episodic (graph) | Neo4j Session nodes | Graph traversal from session | Background (after Stop) |

| | | | |
|---|---|---|---|
| Semantic | Neo4j Memory nodes | FTS + vector similarity | Hot-path + background |
| Procedural | SKILL.md + Neo4j Skill nodes | File load + graph chain | Background + GEPA |
| Working | Context window | Assembled at UserPromptSubmit | Each turn |

## POLE+O Node Labels

| Label | Required Properties | Key Relationships |
|---|---|---|
| Person | name (unique) | PREFERS, KNOWS, PARTICIPATED_IN, HAS_CURRENT_MEMORY |
| Object | name (unique) | USED_IN, DEPENDS_ON, OBSERVED_BY |
| Location | name | LOCATED_IN, HOSTED_AT |
| Event | name, occurred_at | ATTENDED, TRIGGERED, SOLVED_BY |
| Organisation | name (unique) | WORKS_AT, PART_OF |
| Skill | name (unique) | REQUIRES, EVOLVED_FROM, USED_IN |
| Session | session_id (unique) | PARTICIPATED_IN, TOUCHED, HAS_MEMORY |
| Memory | memory_id (unique) | PREVIOUS, HAS_CURRENT_MEMORY, HAS_MEMORY |

## Key Python Patterns

| Task | Function |
|---|---|
| Init Neo4j schema | init_schema(uri, user, password) |
| Create session node | create_session_node(driver, session_id, user, model) |
| Close session | close_session_node(driver, session_id, message_count) |
| Store semantic fact | upsert_semantic_memory(driver, user, fact, confidence) |
| Hybrid retrieval | hybrid_retrieve(driver, user, query, k=8) |
| Entity extraction | extract_entities(messages) → dict |
| Link entities | link_entities_to_session(driver, session_id, entities) |
| Skill sync | sync_skills_to_kg(driver, skills_dir) |
| Skill chain | find_skill_chain(driver, goal) → list[str] |

| | |
|---|---|
| GEPA evolution | run_gepa_cycle(driver, skill_name) |
| GC cleanup | compress_previous_chains(driver, keep=3) |
| GDPR erase | gdpr_erase_user(driver, user_name) — IRREVERSIBLE |

# References

| Resource | URL |
|---|---|
| Neo4j Modeling Agent Memory | https://neo4j.com/blog/developer/modeling-agent-memory/ |
| neo4j-labs/agent-memory | https://github.com/neo4j-labs/agent-memory |
| Hermes Agent (Nous Research) | https://hermes-agent.nousresearch.com/ |
| Claude Agent SDK Docs | https://code.claude.com/docs/en/agent-sdk/agent-loop |
| Neo4j 5.x Vector Index | https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/ |
| Self-Improving Agent (dev.to) | https://dev.to/programmingcentral/beyond-the-context-window-how-to-build-a-self-improving-ai-agent-with-persistent-memory-31lh |